DOCUMENTO TÉCNICO E DIDÁTICO

Como funciona o NodeWatch

Análises de grafos, Machine Learning e limites do sistema: o que existe hoje, como cada análise é feita e o que ainda está planejado.

Projeto	NodeWatch — Sistema de Detecção de Fraudes em Transações Financeiras (Grupo 4)
Disciplina	Teoria dos Grafos — Profa. Dra. Andrea Ono Sakai
Integrantes do projeto	Caio Winkler Marangoni, Guilherme Lombardi, Ryan dos Santos Veloso, Julia Emily Leonardo Barbosa e Guilherme Liborio Camargo (conforme o `README.md`)
Versão documentada	Branch `main` do repositório `1s4ntos/NodeWatch`, commit `03b84d2`
Resultados de ML	Execução de 02/11/2026: artefatos `estado_rede-20261102T083053Z` e `lavagem-20261102T083253Z`
Data	02/11/2026

Sumário

1 Visão geral **3**
1.1 O problema e o que o sistema recebe 3
1.2 Componentes e situação de cada um 3
1.3 Quatro coisas que não se confundem 4
1.4 Como um alerta apoia uma investigação 4

2 Arquitetura do sistema **5**
2.1 Componentes e caminho dos dados 5
2.2 Arquitetura modular do ML 6
2.3 Persistência e carregamento dos modelos 7
2.4 Integração entre fontes: planejada 7

3 Dados utilizados **7**
3.1 Telemetria de rede: dataset_final.csv 7
3.2 Transações sintéticas: IBM AML HI-Small 8
3.3 CSV do módulo de grafos 9
3.4 Dicionário dos atributos principais 10

4 Como as análises são feitas **11**
4.1 Regras determinísticas e modelos aprendidos 11
4.2 Análises de grafos 11
4.3 Indicadores de rede 13
4.4 Atributos financeiros sem olhar o futuro 14
4.5 Motivos verificáveis 15

5 Como o Machine Learning funciona **16**
5.1 Conceitos básicos 16
5.2 Treino, validação, teste e inferência 16
5.3 Tarefa estado_rede 16
5.4 Tarefa lavagem 17
5.5 Algoritmos utilizados 18
5.6 Implementação própria, referência e baseline 18
5.7 Inferência com o pipeline salvo 19

6 Exemplos completos **19**
6.1 Exemplo de rede 19
6.2 Exemplo financeiro 20
6.3 Exemplo do módulo de grafos 21

7 Avaliação e resultados **22**
7.1 Como ler as métricas 22
7.2 Resultados da rede 23
7.3 Resultados da lavagem 24
7.4 Comparações feitas pela auditoria 25
7.5 Reuso do teste e incerteza 26
7.6 Testes automatizados 26

8 Uso e reprodução **27**
8.1 Instalação 27
8.2 Módulo de grafos e painel 27
8.3 Módulo de ML 27
8.4 Como preparar um arquivo compatível 28
8.5 Como carregar o modelo e ler as saídas 28
8.6 Colunas ausentes e dados inválidos 29

9 Limitações e próximos passos **30**
9.1 Limitações verificadas 30
9.2 Pendências das auditorias 30
9.3 Como trocar modelos no futuro 31

10 Glossário **31**

11 Referências **32**

1 Visão geral

> **Como ler este documento.** Cada seção explica primeiro para que serve um componente e depois como ele funciona. As etiquetas implementado, experimental e planejado indicam a situação verificada no código. Os números vêm dos arquivos de resultados versionados em `docs/ml/resultados/` e da auditoria de 02/11/2026, sempre com a fonte indicada. Nenhum modelo foi retreinado para escrever este documento.

1.1 O problema e o que o sistema recebe

O NodeWatch é um MVP (*Minimum Viable Product*, produto mínimo viável) acadêmico para **apontar padrões suspeitos em transações financeiras** e ajudar uma pessoa analista a decidir o que investigar primeiro. A ideia original, da disciplina de Teoria dos Grafos, é tratar contas como vértices e transações como arestas: certos desenhos nesse grafo, como o dinheiro que dá uma volta e retorna à conta de origem, são típicos de lavagem de dinheiro.

Depois foi acrescentado um módulo de ML (*Machine Learning*, aprendizado de máquina) com duas tarefas independentes: prever o estado de uma rede de comunicação a partir de telemetria e pontuar transações de um conjunto sintético de lavagem. O sistema **não prova fraude**. Ele organiza indícios e ordena alertas, e a decisão continua humana.

Tabela 1 — O que entra no sistema

Entrada	Formato	Quem usa
Transações para análise de grafos	CSV (*Comma-Separated Values*) no formato PaySim simplificado: `step, type, amount, nameOrig, nameDest, isFraud`	Módulo de grafos (`src/main.py`)
Telemetria de rede	`dataset_final.csv` (esquema `rede_v1`): uma medição por linha	ML, tarefa `estado_rede`
Transações sintéticas de lavagem	IBM AML HI-Small: `HI-Small_Trans.csv` ou o `.zip` que o contém (esquema `hismall_v1`)	ML, tarefa `lavagem`
Contas e padrões do HI-Small	`HI-Small_accounts.csv` e `HI-Small_Patterns.txt` (opcionais)	Só auditoria e avaliação por tipologia
Perguntas do analista	Texto no chat do painel	Sentinel AI (modelo de linguagem externo)

1.2 Componentes e situação de cada um

O repositório reúne componentes de épocas e propósitos diferentes. A tabela separa o que está implementado, o que é experimental e o que só existe como plano.

Tabela 2 — Componentes verificados no commit 03b84d2

Componente	O que faz	Situação
Análise de grafos (CLI) `src/main.py`, `src/algoritmos/`	Lê o CSV, monta um multigrafo direcionado e executa: ciclos por DFS (*Depth-First Search*, busca em profundidade), SCC (*Strongly Connected Components*, componentes fortemente conectados) por Kosaraju e centralidade de grau com pontuação de risco. Exporta JSON (*JavaScript Object Notation*).	implementado
Detector de fan-out e fan-in `src/algoritmos/fan_detection.py`	Encontra contas que enviam para muitas contas distintas, ou recebem de muitas, numa janela de tempo curta.	experimental biblioteca e script; fora da CLI e do painel
Painel visual `interface/index.html`	Mostra grafo, ciclos, SCC e centralidade do CSV de exemplo. Os resultados estão **pré-calculados e embutidos** no HTML; não há envio de arquivo.	implementado demonstração

Componente	O que faz	Situação
Sentinel AI `src/sentinel_ai.py`	Servidor Flask com um chat que envia a pergunta e o estado do painel a um LLM (*Large Language Model*) externo: Google Gemini por padrão, ou Anthropic. Explica resultados; não detecta nada sozinho.	implementado exige chave de API
ML — `estado_rede` `src/ml/`	Prevê o estado (OK, RISCO ou FALHA) da próxima medição de um fluxo de rede.	implementado treinado e avaliado
ML — `lavagem` `src/ml/`	Pontua transações do HI-Small sintético quanto à chance de fazerem parte de lavagem.	implementado só em dados sintéticos
ML — `fusao_rede_transacao`	Usaria sinais de rede para pontuar transações. Treinar falha de propósito e lista o que falta.	planejado bloqueado no código
XGBoost	Avaliado pela auditoria de 02/11/2026 em scripts isolados. Não está no código nem nas dependências do NodeWatch.	avaliado, não adotado

1.3 Quatro coisas que não se confundem

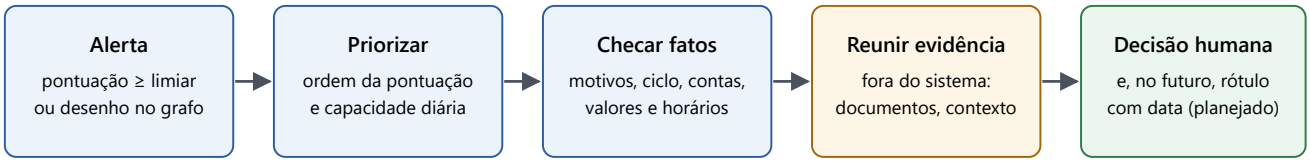
Os termos abaixo aparecem em todo o projeto e têm sentidos diferentes. Misturá-los leva a conclusões erradas, como tratar uma lentidão de rede como prova de fraude.

Tabela 3 — Termos e o que cada um não é

Termo	O que significa no NodeWatch	Quem produz	O que não é
Anomalia de rede	Previsão de que a próxima medição de um fluxo de rede estará em RISCO ou FALHA, isto é, com latência, jitter ou perda de pacotes acima das referências do fluxo.	Cabeça `estado_rede`	Não é indício de fraude. Hoje não existe nenhuma ligação entre a rede e as transações.
Comportamento financeiro suspeito	Desenho no grafo de transações (ciclo, grupo fortemente conectado, conta concentradora, fan-out) ou fato verificável sobre uma transação (por exemplo, primeira transação entre duas contas).	Módulo de grafos; motivos da cabeça `lavagem`	Não é ilícito por si: folhas de pagamento e comércios também criam hubs e ciclos.
Previsão de lavagem	`score_lavagem` igual ou acima do limiar, aprendido nas transações sintéticas do HI-Small.	Cabeça `lavagem`	Não é probabilidade calibrada nem confirmação de fraude; não vale automaticamente para dados reais.
Fraude confirmada	Conclusão de uma investigação humana, com evidência e data de confirmação.	Nenhum componente	Nenhum dado do projeto tem esse rótulo. O contrato proposto `transacao_v1` prevê `rotulo_fraude` com origem e data.

1.4 Como um alerta apoia uma investigação

Um alerta é um pedido de atenção, não um veredito. O fluxo esperado é o da Figura 1. Na lavagem, por exemplo, investigar as 1.000 transações de maior pontuação de cada dia do teste teria encontrado de 54% a 62% da lavagem daquele dia (seção 7.3). A pontuação serve para **ordenar a fila** de trabalho; os motivos e os desenhos do grafo servem para **começar a checagem**.



Azul: o que o NodeWatch faz. Âmbar e verde: trabalho da pessoa analista.

Figura 1 — Papel de um alerta numa investigação. O sistema entrega as três primeiras etapas.

2 Arquitetura do sistema

2.1 Componentes e caminho dos dados

O NodeWatch tem três caminhos que **não trocam dados entre si**. A Figura 2 mostra cada um da entrada à saída, com o arquivo responsável por cada etapa. As duas tarefas de ML ficam guardadas no mesmo artefato, mas cada uma é treinada e avaliada só com os próprios dados.

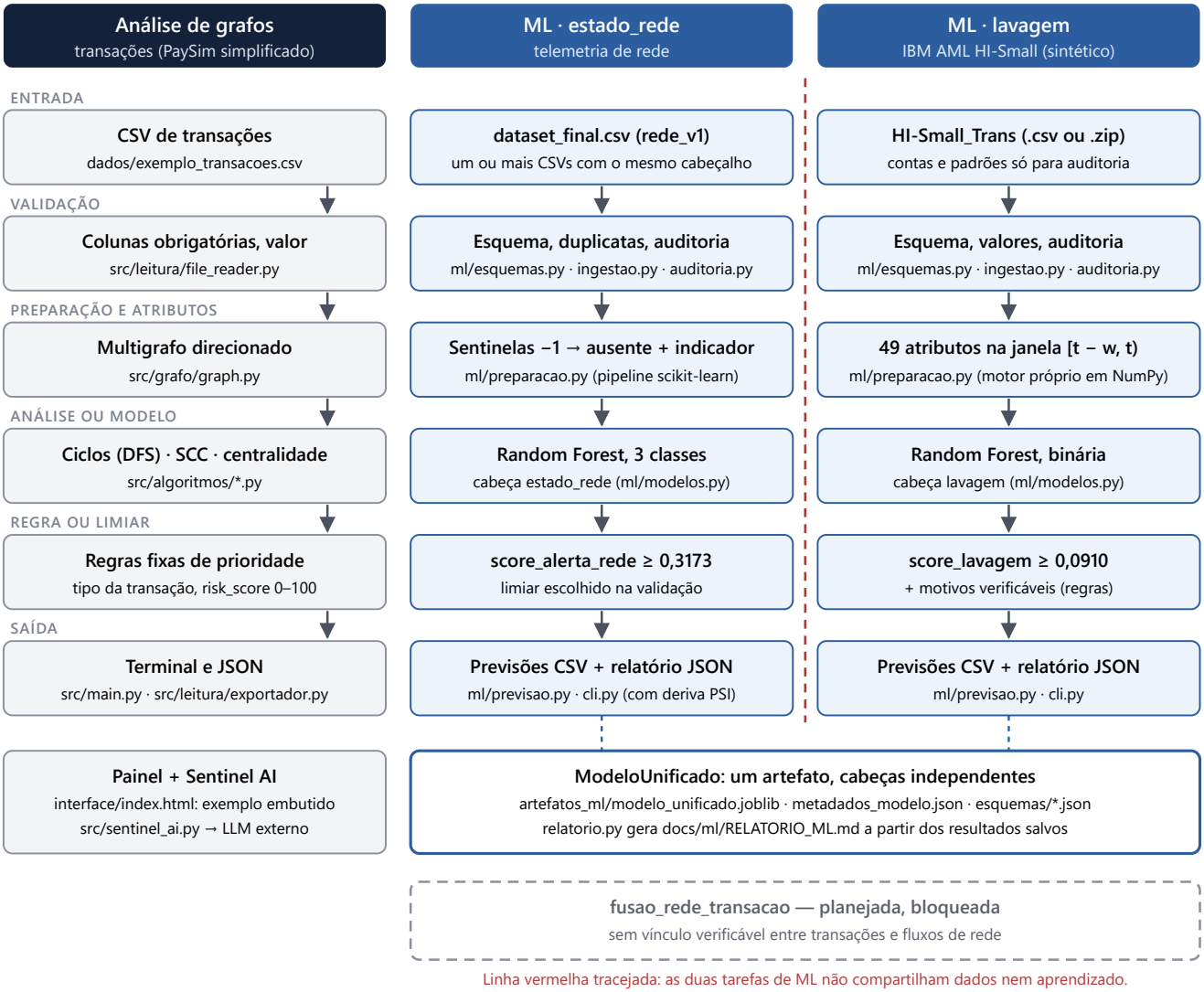


Figura 2 — Os três caminhos do NodeWatch. Caixas cinza: módulo de grafos. Caixas azuis: módulo de ML. Contorno tracejado: componente planejado.

Tabela 4 — Etapas e módulos responsáveis

Etapa	Grafos	ML · estado_rede	ML · lavagem
Entrada	Um CSV (`--input`)	Um ou mais CSVs `rede_v1`	Um CSV ou `.zip` HI-Small; histórico opcional (`--historico`)
Validação	`file_reader.py`: cabeçalho e `amount` numérico	`esquemas.py` e `ingestao.py`: tipos, faixas, NaN e infinitos, duplicatas	`ingestao.py`: cabeçalho exato, ausentes, valores negativos ou não finitos
Preparação	`graph.py`: vértices e arestas	`preparacao.py`: sentinelas, indicadores, remoção de constantes	`preparacao.py`: câmbio, 49 atributos que só olham o passado
Análise	`cycle_detection.py`, `scc.py`, `centralidade.py`	Random Forest (`modelos.py`)	Random Forest (`modelos.py`)
Regra ou limiar	Prioridade pelo tipo; `risk_score` por regra fixa	Alerta se P(RISCO) + P(FALHA) ≥ 0,3173	Alerta se `score_lavagem` ≥ 0,0910; motivos
Saída	Texto no terminal; JSON em `dados/analises/`	CSV de previsões e `.relatorio.json`	CSV de previsões e `.relatorio.json`
Persistência	Nenhuma (recalcula a cada execução)	`artefatos_ml/modelo_unificado.joblib` e metadados (seção 2.3)	

2.2 Arquitetura modular do ML

Para que serve. A arquitetura modular permite guardar várias tarefas num único artefato, com uma única forma de uso, sem acoplar tarefas que não têm dados em comum. Trocar o modelo de uma tarefa não exige mexer na outra.

Como funciona. O objeto `ModeloUnificado` (`src/ml/modelos.py`) tem duas coleções:

- **codificadores**, um por fonte de dados. Guardam o que foi aprendido sobre a entrada: para a rede, o esquema, os preditores e as sentinelas; para o financeiro, o `PreparadorFinanceiro`, com câmbio, formatos, moedas, taxas de lavagem por formato e quantis do treino;
- **cabeças**, uma por tarefa. Cada `CabecaClassificacao` guarda um pipeline do scikit-learn já ajustado, as classes, a lista exata de colunas de entrada, a família e os parâmetros do modelo, o limiar de alerta e os metadados do treino.

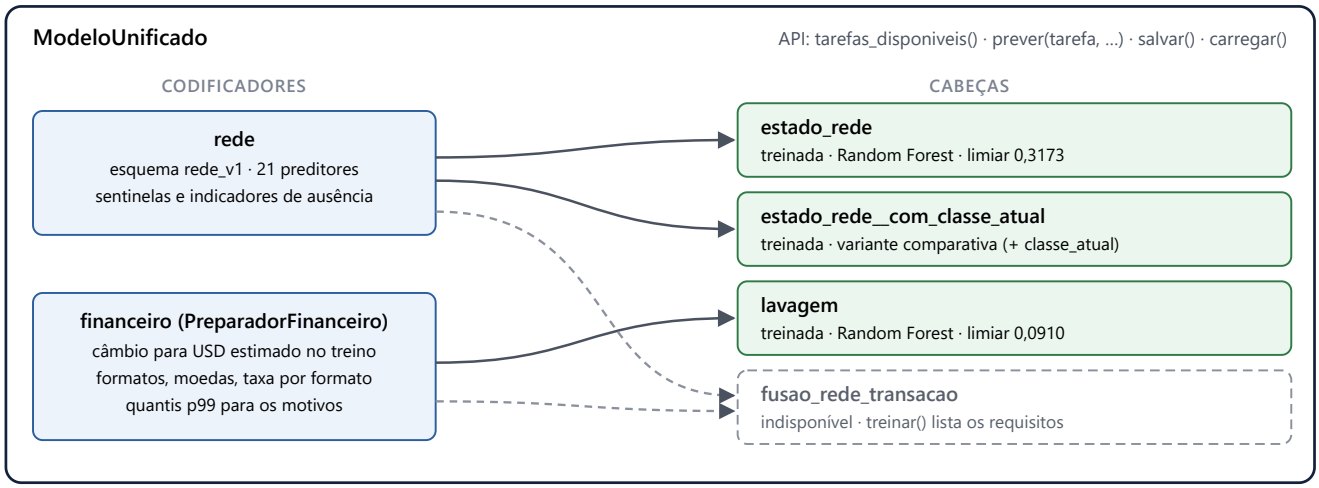


Figura 3 — Estrutura do `ModeloUnificado`. Setas cheias: dados usados no treino. Setas tracejadas: a ligação que a fusão exigiria e que ainda não existe.

"Arquitetura única" quer dizer **um artefato e uma interface**, não um modelo conjunto. Não há camada compartilhada que aprenda com as duas fontes: os dados não têm nenhuma amostra em comum, então tal camada não teria o que aprender.

2.3 Persistência e carregamento dos modelos

Ao fim de `treinar`, o `ModeloUnificado` é salvo com `joblib` (compressão nível 3) em `artefatos_ml/modelo_unificado.joblib`. Junto saem `metadados_modelo.json`, com tarefas, colunas, limiares, critérios de escolha, versões das bibliotecas e hash dos dados, e os esquemas em `artefatos_ml/esquemas/`.

- `ModeloUnificado.carregar()` confere se o arquivo contém mesmo um `ModeloUnificado` e emite um aviso se a versão do scikit-learn atual for diferente da usada no treino.
- A pasta `artefatos_ml/` é ignorada pelo git, por ser volumosa. Cópias dos resultados ficam versionadas em `docs/ml/resultados/`, com os caminhos da máquina trocados pelo nome do arquivo.
- **Segurança:** um arquivo `joblib` executa código ao ser carregado. Carregue apenas artefatos gerados pela própria equipe.

2.4 Integração entre fontes: planejada

A pergunta natural é se uma anomalia de rede ajuda a detectar fraude. Hoje ela não pode ser respondida: as transações do HI-Small são de setembro de 2022, a telemetria é de setembro e outubro de 2026, e não existe chave que ligue uma transação a um fluxo de rede. Proximidade de horário não é vínculo.

Por isso a cabeça `fusao_rede_transacao` recusa o treino e lista o que falta:

1. transações no esquema `transacao_v1` com `timestamp_disponivel`;
2. tabela `vinculo_v1` que ligue a transação, ou uma chave técnica como sessão ou dispositivo, a um `id_fluxo`, com validade no tempo e fonte documentada;
3. telemetria do mesmo período das transações;
4. previsões de rede fora da amostra, de um modelo treinado antes de cada transação;
5. rótulo de fraude com origem e data de confirmação.

O código de junção já existe e é testado com dados sintéticos (`src/ml/integracao.py`). Ele usa a junção temporal "as of" para trás: cada transação recebe o último sinal de rede emitido antes dela, dentro de uma tolerância (480 s por padrão). Sinais futuros e sinais de modelos treinados depois da transação são descartados. O contrato está em `docs/ml/CONTRATO_INTEGRACAO.md`.

3 Dados utilizados

3.1 Telemetria de rede: dataset_final.csv

Origem. Arquivo de telemetria do material da disciplina, também usado no notebook didático da professora. O código que o gerou não foi fornecido; por isso parte do dicionário é marcada como "inferida". A identidade do arquivo é conferida pelo SHA-256 `125453f1…b997` (valor completo em `docs/ml/README.md`).

Unidade. Cada linha é **uma medição de um fluxo**: um par sonda → âncora (`id_fluxo` = `probe_id_anchor_id`) num instante (`timestamp`, segundos Unix, fuso presumido UTC). As medições de um fluxo vêm a cada 240 s em mediana, de forma irregular.

Tabela 5 — Números do dataset_final.csv

Item	Valor	Item	Valor
Linhas	333.954	Treino (28/09 a 03/10/2026)	241.359
Fluxos / sondas / âncoras	135 / 129 / 30	Validação (04/10/2026)	46.490
NaN, infinitos, duplicatas	0 / 0 / 0	Teste (05/10/2026)	46.105
Alvo OK / RISCO / FALHA	87,5% / 8,7% / 3,7%	Menor lacuna entre períodos	3.605 s

Fonte: `docs/ml/resultados/auditoria_rede.json` e `docs/ml/RELATORIO_ML.md`, seção 1.1.

Rótulos. Há duas colunas de estado:

- `classe_atual`: estado da medição atual, conhecido no instante da previsão;
- `alvo`: estado da **próxima** medição do mesmo fluxo. É o que o modelo aprende a prever.

A auditoria descobriu uma regra de limiares que reproduz `classe_atual` em 99,94% das linhas (`src/ml/auditoria.py`):

- **FALHA** se não houve RTT (`rtt_ausente` = 1), se o RTT passou do p99 de referência (`razao_rtt_p99` > 1) ou se a perda foi de 50% ou mais;
- senão, **RISCO** se o RTT ou o jitter passaram do p95 de referência, ou se houve qualquer perda;
- senão, **OK**.

Ou seja, o rótulo é uma regra aplicada à medição seguinte. A tarefa de ML é antecipar essa regra uma medição à frente.

> **Limitações do arquivo.**
>
> - O período sobre o qual foram calculados os percentis de referência (p95 e p99) não está documentado. No treino, 7,5% das medições ficam acima do "p95", e não 5%.
> - Seis linhas têm `alvo` diferente da `classe_atual` seguinte. Todas vêm logo antes de uma lacuna na série, o que sugere que o alvo foi calculado antes de linhas serem removidas. Elas foram mantidas.
> - Os mesmos 135 fluxos aparecem nos três períodos. O teste mede o futuro de fluxos conhecidos; fluxos novos têm avaliação própria (seção 7.2).

3.2 Transações sintéticas: IBM AML HI-Small

Origem. Conjunto **sintético** publicado pela IBM e descrito por Altman et al. (2023), gerado por um simulador de agentes que inclui tipologias de lavagem. "HI" indica a variante com taxa de lavagem mais alta; "Small" é o menor tamanho. Os arquivos chegaram ao grupo num `.rar` entregue pela professora. **Os registros não são transações bancárias reais**, e as contas são identificadores fictícios.

Unidade. Cada linha é uma transação simulada de uma conta (banco + conta) para outra, com horário em resolução de minuto.

Tabela 6 — Números do HI-Small e divisão temporal

Período	Dias (2022)	Transações	Lavagem	Taxa	Uso
Treino	01 a 06/09	3.248.921	2.530	0,08%	ajuste dos modelos
Validação	07 e 08/09	965.524	1.036	0,11%	escolha do modelo e do limiar
Teste	09 e 10/09	862.792	956	0,11%	medição final
Cauda	11 a 18/09	1.108	655	59,1%	só reportada à parte
Total	**01 a 18/09**	**5.078.345**	**5.177**	**0,10%**	**515.088 contas**

Fonte: `docs/ml/resultados/auditoria_financeiro.json` e `docs/ml/RELATORIO_ML.md`, seção 2.1.

Campos. `Timestamp`, `From Bank`, `Account`, `To Bank`, `Account` (o nome aparece duas vezes no cabeçalho: origem e destino), `Amount Received`, `Receiving Currency`, `Amount Paid`, `Payment Currency`, `Payment Format` e o rótulo `Is Laundering` (1 = a transação faz parte de lavagem segundo o simulador).

Arquivos auxiliares. `HI-Small_accounts.csv` descreve as entidades donas das contas, e `HI-Small_Patterns.txt` lista 370 tentativas de lavagem em 8 tipologias. Os dois servem só para auditoria e avaliação; **nunca viram atributo**, porque o arquivo de padrões é a própria resposta. A auditoria mediu que os atributos de conta não revelam o rótulo: AP de validação de 0,0011 a 0,0017, contra prevalência de 0,0011.

> **Limitações do conjunto.**
>
> - **Cauda artificial:** de 11/09 em diante quase não há atividade normal, e 59% das transações são lavagem. Esse trecho fica fora das métricas principais.
> - **Sinal do gerador:** o formato ACH concentra 87% da lavagem (4.483 de 5.177), e Wire e Reinvestment não têm nenhum caso. Isso pode ser um artefato do simulador, por isso há um baseline que usa só o formato.
> - **Sem padrão:** 1.968 das 5.177 transações de lavagem não aparecem em nenhum padrão de `HI-Small_Patterns.txt`.
> - **Tentativas longas:** 206 das 370 tentativas atravessam a fronteira entre períodos.
> - **Sem confirmação:** o rótulo não tem data de confirmação, e o fuso do horário não é documentado (assume-se UTC).

3.3 CSV do módulo de grafos

O módulo de grafos lê um CSV no formato do simulador PaySim (Lopez-Rojas et al., 2016), simplificado. Só `nameOrig`, `nameDest` e `amount` são obrigatórias:

Coluna	No grafo	Uso
`nameOrig` / `nameDest`	Vértice de origem e de destino	Estrutura do grafo
`amount`	Peso da aresta	Valor do ciclo, volumes e volume interno do SCC
`type`	Metadado da aresta	Prioridade do ciclo e `risk_score` (TRANSFER, CASH_OUT, PAYMENT, CASH_IN, DEBIT)
`step`	Instante da aresta (opcional)	Só o detector de fan-out e fan-in usa
`isFraud`	Rótulo auxiliar (opcional)	Estatísticas do JSON e do painel; os detectores não o usam

O exemplo versionado, `dados/exemplo_transacoes.csv`, tem 16 transações entre 15 contas e foi montado à mão para conter três ciclos (seção 6.3).

3.4 Dicionário dos atributos principais

O dicionário completo, com tipo, unidade, sentinela e status (verificado ou inferido), é gerado do código em `docs/ml/DICIONARIO_DADOS.md`. Aqui ficam os atributos mais importantes para entender as análises.

Tabela 7 — Atributos de rede (rede_v1)

Nome técnico	Significado	Uso
`perda_pct`	Perda de pacotes na medição, em %.	Preditor; regra do rótulo
`rtt_ausente`	1 se a medição não obteve RTT (*Round-Trip Time*, tempo de ida e volta).	Preditor; regra do rótulo
`razao_rtt_p95`, `razao_rtt_p99`	RTT da medição dividido pelo p95 ou p99 de referência do fluxo. Acima de 1 = mais lento que o habitual. −1 quando não há RTT.	Preditor; regra do rótulo
`razao_jitter_p95`	Jitter (variação do atraso) dividido pelo p95 de referência do fluxo.	Preditor; regra do rótulo
`amplitude_rtt_rel`	Amplitude do RTT na medição, relativa ao RTT típico.	Preditor
`delta_ttl_clip`	Variação do TTL (*Time To Live*) em relação à medição anterior, recortada; negativos são legítimos e indicam mudança de rota.	Preditor
`perda_media_w3/w6/w12`	Média da perda nas últimas 3, 6 ou 12 medições do fluxo, incluindo a atual.	Preditor (histórico)
`rtt_p95_max_wk`, `jitter_p95_max_wk`	Maior razão de RTT ou de jitter nas últimas k medições.	Preditor (histórico)
`n_rtt_acima_p95_wk`	Quantas das últimas k medições tiveram RTT acima do p95.	Preditor (histórico)
`perdas_consecutivas`	Medições seguidas com perda até a atual.	Preditor
`sem_rtt_w3/w6/w12`	Criados pelo pipeline: 1 se nenhuma medição da janela teve RTT.	Preditor derivado
`classe_atual`	Estado da medição atual.	Baseline de persistência; só a variante comparativa a usa como preditor
`alvo`	Estado da próxima medição.	Alvo; nunca preditor

Tabela 8 — Atributos financeiros calculados pelo NodeWatch (49 no total)

Nome técnico	Significado	Janela
`log_valor_usd`	log(1 + valor pago convertido para dólar americano).	transação atual
`n_env_origem_6h/24h/72h`	Quantas transações a conta de origem enviou antes.	6, 24 ou 72 h
`n_env_origem_total`	Quantas transações a origem já enviou.	todo o passado
`soma_env_origem_usd_24h`	Total enviado pela origem, em USD.	24 h
`z_valor_origem`	Quão atípico é o valor atual frente aos envios anteriores da origem (seção 4.4).	todo o passado
`n_rec_origem_24h`, `soma_rec_origem_usd_24h`	Quanto a origem recebeu: contagem e total em USD.	24 h
`razao_saida_entrada_24h`	Total enviado ÷ (total recebido + 1), em USD (seção 4.4).	24 h
`min_desde_env_origem`, `min_desde_rec_origem`	Minutos desde o último envio e o último recebimento da origem (teto de 10 dias).	todo o passado
`n_env_destino_24h`, `n_rec_destino_24h/72h`, `min_desde_rec_destino`	Atividade recente da conta de destino.	24 ou 72 h

Nome técnico	Significado	Janela
`par_novo`, `n_par_previas`	Se é a primeira transação entre essa origem e esse destino; quantas houve antes.	todo o passado
`dest_distintos_origem`, `orig_distintas_destino`	Contrapartes distintas acumuladas pela origem e pelo destino.	todo o passado
`moeda_diferente`, `mesmo_banco`, `auto_transferencia`, `sem_taxa_cambio`	Indicadores da própria transação.	transação atual
`sem_historico_*` (3)	1 se a conta ainda não tinha o evento correspondente.	todo o passado
`formato_*` (7), `moeda_*` (15)	Formato de pagamento e moeda de pagamento em codificação one-hot.	transação atual

Fonte: `src/ml/preparacao.py` (`atributos_financeiros`) e lista de colunas em `docs/ml/resultados/treino_lavagem.json`.

4 Como as análises são feitas

4.1 Regras determinísticas e modelos aprendidos

O NodeWatch usa dois tipos de análise, e a distinção importa para interpretar resultados:

- **Regra determinística:** escrita por pessoas, com parâmetros fixos. A mesma entrada sempre gera a mesma saída, e é possível explicar cada resultado passo a passo. Não depende de rótulos.
- **Modelo aprendido:** seus parâmetros são estimados a partir de exemplos rotulados. Pode capturar combinações que ninguém escreveu, mas só é confiável dentro do tipo de dado em que foi avaliado.

Tipo	Onde aparece no NodeWatch
Regras determinísticas	Ciclos, SCC, centralidade e `risk_score`; detector de fan-out e fan-in; motivos verificáveis da lavagem; baselines (persistência, taxa de lavagem do formato, prevalência, classe majoritária).
Regra embutida nos dados	A regra de limiares que gera `classe_atual` e, uma medição depois, o `alvo` da rede.
Modelos aprendidos	As cabeças `estado_rede` e `lavagem` (Random Forest escolhida; regressão logística e HistGradientBoosting foram candidatas).

4.2 Análises de grafos

Todas as análises desta seção são regras determinísticas sobre o **multigrafo direcionado ponderado** montado por `src/grafo/graph.py`. Cada conta é um vértice, cada transação é uma aresta da origem para o destino com peso igual ao valor, e pode haver várias arestas entre o mesmo par. Nenhuma delas usa a ordem do tempo, exceto o detector de fan-out e fan-in.

Ciclos por busca em profundidade

Entrada	O grafo inteiro do arquivo.
Cálculo	`find_cycles` (`src/algoritmos/cycle_detection.py`): 1. restringe as arestas às que ligam vértices do mesmo SCC, porque todo ciclo fica dentro de um SCC; 2. a partir de cada vértice, em ordem, faz uma DFS iterativa e registra os **ciclos simples**, em que nenhuma conta se repete. Cada ciclo é registrado uma única vez, na busca que parte do seu menor vértice; 3. dá categoria ANOMALIA a laços de uma conta para ela mesma e CICLO aos demais; 4. a prioridade é o tipo de maior risco entre as arestas: TRANSFER = ALTO, CASH_OUT = MÉDIO-ALTO, PAYMENT = MÉDIO, CASH_IN ou DEBIT = BAIXO; 5. calcula o valor total, a "perda" (valor da primeira aresta menos o da última) e o número de intermediárias.
Janela	Nenhuma. O ciclo é estrutural: as transações podem ter acontecido em qualquer ordem no tempo.
Saída	Lista de `CycleResult` (caminho, categoria, prioridade, valor, perda, intermediárias).
Interpretação	Dinheiro que sai, passa por intermediárias e volta à origem, como na fase de ocultação da lavagem (*layering*).
Limitações	• Enumerar todos os ciclos simples tem pior caso exponencial no número de contas. • A ordem de saída deveria ser prioridade → valor → intermediárias, mas a prioridade não entra na ordenação: a chave procura um número num dicionário de textos (defeito A1, aberto). • Quando há várias transações entre o mesmo par, só a primeira entra no valor e na prioridade (defeito A3, aberto).

Componentes fortemente conectados (Kosaraju)

Entrada	O grafo inteiro do arquivo.
Cálculo	`find_scc` (`src/algoritmos/scc.py`), em duas passagens de DFS: 1. no grafo original, empilha os vértices na ordem em que terminam; 2. no grafo com as arestas invertidas, desempilha: cada busca completa descobre um componente. Custo O(V + E). Componentes com mais de uma conta são marcados como suspeitos. O volume interno soma todas as transações dentro do grupo.
Janela	Nenhuma.
Saída	Lista de `SCCResult` (contas, tamanho, volume interno, arestas internas), suspeitos primeiro.
Interpretação	Um grupo em que o dinheiro pode circular entre todas as contas, o que vai além de um ciclo isolado.
Limitação	Conexão não é ilicitude: redes legítimas também formam grupos fortemente conectados.

Centralidade de grau e pontuação de risco

Entrada	O grafo e a lista de ciclos.
Cálculo	`compute_centrality` (`src/algoritmos/centralidade.py`): • conta as transações recebidas (grau de entrada) e enviadas (grau de saída), soma os volumes e conta em quantos ciclos a conta aparece; • `risk_score` = pontos pelo tipo mais enviado (TRANSFER 40, CASH_OUT 30, PAYMENT 15, CASH_IN e DEBIT 10; 0 se não envia) + 15 por ciclo, com teto 100; • `hub_type`: SUSPEITO se está em algum ciclo; DISTRIBUIDOR se envia 3 vezes ou mais do que recebe; COLETOR se envia um terço ou menos; INTERMEDIÁRIO se o grau total é 4 ou mais; NORMAL nos demais casos.
Janela	Nenhuma.
Saída	Lista de `AccountCentrality` ordenada por `risk_score`.
Interpretação	Destaca contas distribuidoras, coletoras e de passagem.
Limitações	Os pesos são fixos e não foram calibrados com dados. Quando dois tipos empatam como "mais enviado", o escolhido depende da ordem interna de um conjunto do Python, e a pontuação pode mudar entre execuções (defeito A2, aberto).

Fan-out e fan-in por janela de tempo experimental

Entrada	O grafo com `step` preenchido.
Cálculo	`find_fan_patterns` (`src/algoritmos/fan_detection.py`) ordena as transações de cada conta por `step` e desliza uma janela, procurando o trecho com mais contrapartes distintas. Padrão: 10 contrapartes ou mais (`min_counterparts`). Contas com contrapartes demais no período inteiro podem ser ignoradas (`max_total_counterparts`).
Janela	Duas transações estão na mesma janela se a diferença de `step` é de até 72 (horas no PaySim).
Saída	Lista de `FanResult`, com no máximo um por conta e tipo.
Interpretação	Dispersão (fan-out) ou concentração (fan-in) rápida de recursos.
Limitações	Não está ligado à CLI nem ao painel; é usado por testes e por `scripts/benchmark_fan.py`. Contas legítimas de alto volume disparam o padrão. `add_edges_from` descarta o `step` ao copiar arestas (defeito A5, aberto).

A CLI pode ainda **exportar tudo em JSON** (`--export-json`) para `dados/analises/`, sem sobrescrever análises anteriores. O painel mostra o mesmo tipo de resultado, mas a partir de dados embutidos no HTML; o Sentinel AI responde perguntas com base nesse estado e não altera nenhum cálculo.

4.3 Indicadores de rede

Para que servem. Descrevem a qualidade de um caminho de rede (latência, variação e perda) e como ela vinha evoluindo nas medições anteriores.

Entrada	Uma linha `rede_v1`. Os indicadores já chegam calculados no CSV; o NodeWatch não mede a rede.
Cálculo	Na origem, as razões dividem o valor da medição pelo percentil de referência do fluxo. No NodeWatch, o pipeline: 1. troca o −1 por ausente nas 13 colunas em que ele significa "sem RTT"; 2. cria um indicador `sem_rtt_wk` por janela; 3. remove colunas constantes (`ttl_reset`); 4. só para a regressão logística, imputa a mediana, aplica sinal(x)·log(1 + |x|) e padroniza. Tudo é ajustado só no treino. Em `delta_ttl_clip`, valores negativos são legítimos e não são tratados como sentinela.

Janela	As últimas 3, 6 ou 12 medições do mesmo fluxo, incluindo a atual, contadas em medições e não em tempo. A auditoria confirmou que as janelas só olham para trás: coincidem com a rolagem para trás em 99,4% a 100% das linhas, contra 4% a 21% olhando para frente.
Saída	A matriz de atributos que entra no modelo.
Interpretação	Razão acima de 1 significa pior que o habitual daquele fluxo; contagens nas janelas mostram se o problema é isolado ou persistente.
Limitação	O período de referência dos percentis não está documentado (seção 3.1).

4.4 Atributos financeiros sem olhar o futuro

Para que servem. Uma transação isolada diz pouco. Os atributos descrevem o **histórico das contas antes da transação**: quanto a origem costuma enviar, se o destino é novo, se o dinheiro acabou de chegar. Para que a avaliação seja honesta, cada atributo só pode usar o que já tinha acontecido. Esse cuidado se chama *point-in-time*.

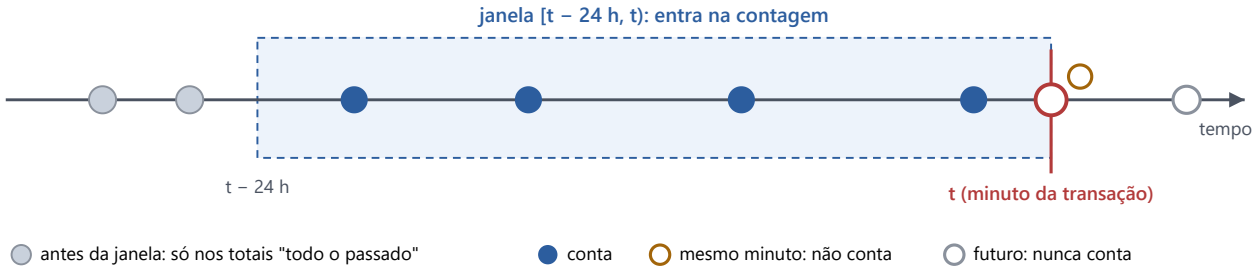


Figura 4 — Janela de um atributo de 24 h para a transação no minuto t. O limite superior é aberto: o próprio minuto t fica de fora, porque o horário tem resolução de minuto e a ordem dentro do minuto não é confiável.

Como é calculado. Contar transações em janelas para 5 milhões de linhas, uma a uma, seria lento demais. O motor próprio, em `src/ml/preparacao.py` (`_IndiceEventos`), resolve com três operações do NumPy:

1. **ordenação** dos eventos por (conta, minuto), com `lexsort`;
2. **busca binária** dos limites da janela de cada consulta, com `searchsorted`: o fim da janela é o primeiro evento da conta no minuto t, e o início é o primeiro evento no minuto t − w;
3. **somas por prefixo** (`cumsum`): a soma dos valores na janela é a diferença entre dois prefixos.

Cada consulta custa O(log n), e a quantidade de eventos numa janela é a diferença entre os dois limites. Antes do cálculo, o valor de cada moeda é convertido para dólar pela mediana de recebido ÷ pago nas transações do treino entre moedas diferentes (15 moedas, 193 pares de moedas).

z_valor_origem e o tratamento da variância

Mede quantos desvios-padrão o valor atual (em log) está acima ou abaixo da média dos envios anteriores da mesma conta:

$$z = (x - \text{média}) \div \sqrt{(\text{variância} + 0{,}01)} \quad \text{com } x = \log(1 + \text{valor em USD})$$

A média e a variância usam todos os envios anteriores da origem. Com menos de dois envios anteriores, z vale 0.

A versão auditada em 02/11/2026 tinha dois problemas numéricos, corrigidos no mesmo dia:

- **Cancelamento.** A variância era calculada como $E[x^2] - E[x]^2$ sobre somas acumuladas de todo o histórico. Com somas grandes, a subtração perdia precisão. *Correção:* antes de acumular, o log-valor de cada conta é deslocado pelo de sua primeira transação enviada. Isso não muda a média das diferenças nem a variância, mas mantém as somas pequenas.

- **Variância quase zero.** Uma conta que enviou poucas vezes valores quase iguais tinha variância ≈ 0, e |z| chegava a 1.939. *Correção:* piso de 0,01 na variância, o que equivale a um desvio mínimo de 0,1 em log, cerca de 10% no valor. Agora |z| fica limitado.

Verificação: depois da correção, o atributo coincide com uma implementação por força bruta nas mesmas 2.000 transações reais do HI-Small em que antes divergia, e testes automatizados cobrem os dois casos. Com o atributo corrigido e o modelo retreinado, a AP de validação da lavagem foi de 0,495 para 0,499. É uma diferença pequena, sem teste estatístico que a confirme como melhoria.

razao_saida_entrada_24h: divergência pendente

$$\text{razão} = (\text{total enviado pela origem em 24 h}) \div (\text{total recebido pela origem em 24 h} + 1) \quad \text{em USD}$$

Valores perto de 1 indicam que a conta repassa o que recebe; valores altos indicam saída maior que a entrada recente. O "+1" evita divisão por zero.

> **Ainda não corrigido.** Em 8 de 2.000 transações reais, o valor difere da força bruta em até 0,7%. Isso ocorre quando as entradas em 24 h somam centavos. As somas vêm de prefixos acumulados sobre todo o histórico, perto de 1,8 trilhão de dólares, e por isso carregam erro absoluto da ordem de um milésimo de dólar. A correção prevista, acumular as somas por conta, não foi feita.

4.5 Motivos verificáveis

Quando uma transação gera alerta de lavagem, a saída traz `motivos_verificaveis`: fatos simples que a pessoa analista pode conferir. Eles são calculados por regras fixas, com quantis do treino, e **não explicam a decisão do modelo**. O modelo pode ter alertado por uma combinação que nenhum motivo descreve.

Motivo	Regra
formato com taxa de lavagem no treino ≥ 5× a média	Taxa de lavagem do formato no treino ÷ taxa geral ≥ 5. Na prática, só ACH (0,58% contra 0,078%).
primeira transação entre origem e destino	`par_novo` = 1
envios da origem em 24 h acima do p99 do treino	`n_env_origem_24h` > 19.667
destino com mais origens distintas que o p99 do treino	`orig_distintas_destino` > 8
origem com mais destinos distintos que o p99 do treino	`dest_distintos_origem` > 14.230
valor atípico para a origem (|z| > 3)	|`z_valor_origem`| > 3
origem recebeu fundos há menos de 60 min	`min_desde_rec_origem` < 60
moedas de pagamento e recebimento diferentes	`moeda_diferente` = 1

Fonte: `PreparadorFinanceiro.motivos` em `src/ml/preparacao.py`; quantis lidos do artefato de 02/11/2026.

Limitação. Os p99 de contagem são dominados por poucas contas de altíssimo volume (19.667 envios em 24 h; 14.230 destinos distintos). Por isso esses dois motivos quase nunca aparecem. Revisar esses limiares é uma pendência (seção 9.2).

5 Como o Machine Learning funciona

5.1 Conceitos básicos

- **Atributos (features)** são as informações de entrada de cada exemplo: as colunas das Tabelas 7 e 8.
- **Alvo** é o que se quer prever: o estado da próxima medição (OK, RISCO ou FALHA) ou se a transação é lavagem (0 ou 1).
- **Classificador supervisionado** é um algoritmo que aprende, a partir de exemplos com o alvo conhecido, uma função que leva atributos a uma classe. Os classificadores daqui também estimam um número entre 0 e 1 para cada classe; esse número é chamado de probabilidade estimada ou pontuação.
- **ML não é sinônimo de rede neural.** Nenhum modelo do NodeWatch é uma rede neural. Os modelos são árvores de decisão combinadas (Random Forest e gradient boosting) e um modelo linear (regressão logística).

5.2 Treino, validação, teste e inferência

Os dados são divididos **no tempo**, para imitar o uso real: o modelo aprende com o passado e é medido num período posterior que não viu.

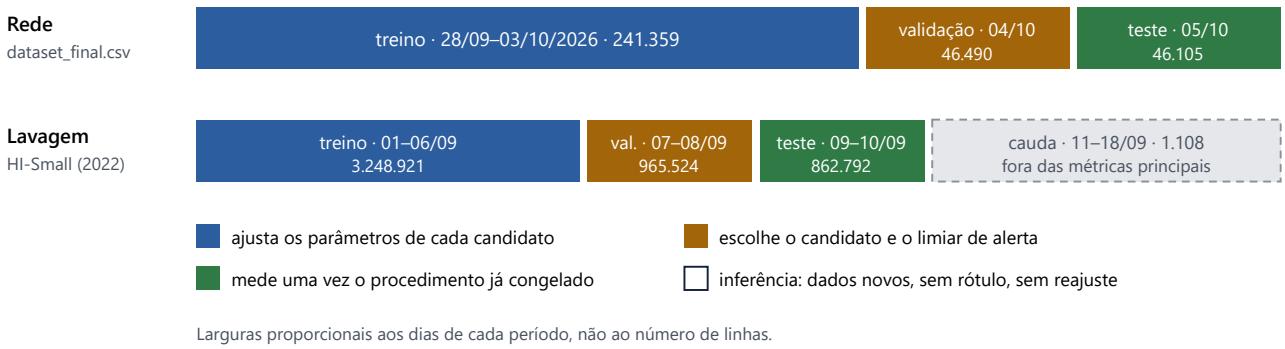


Figura 5 — Divisão temporal das duas tarefas. Na lavagem, os períodos são definidos em `ml_config` (`treino_ate`, `validacao_ate`, `teste_ate`); na rede, pela coluna `split` do arquivo.

Fase	O que acontece	Comando e onde está
Treino	Cada candidato da grade é ajustado só com o período de treino. Imputação, escala e câmbio também são ajustados nesse período.	`treinar` · `experimentos.treinar_*`
Validação	Os candidatos são comparados por uma métrica fixada antes; vence o melhor. O limiar de alerta também é escolhido aqui. O procedimento é então congelado e salvo.	`treinar` · mesmo arquivo
Teste	O artefato salvo é aplicado ao período de teste uma vez, sem reajuste, e as métricas finais são registradas.	`avaliar` · `experimentos.avaliar_*`
Inferência	O artefato é aplicado a arquivos novos, sem rótulo. Nada é reajustado.	`prever` · `src/ml/previsao.py`

Uma exceção documentada: a análise de fluxos e âncoras inéditos (seção 7.2) reajusta a configuração já congelada em subconjuntos do treino, sem nova busca.

5.3 Tarefa estado_rede

Objetivo: prever o estado da próxima medição de cada fluxo, para emitir menos alertas falsos que a simples repetição do estado atual.

- **Alvo:** `alvo` (OK, RISCO ou FALHA).

- **Atributos:** as 21 colunas da Tabela 7, sem `classe_atual`. O pipeline acrescenta 3 indicadores e remove `ttl_reset`, que é constante, então o modelo recebe 23 colunas.
- **Colunas excluídas e por quê:**
  - `alvo`: é a resposta;
  - `split`: é controle do experimento;
  - `id_fluxo`, `anchor_id` e `probe_id`: o modelo decoraria fluxos em vez de aprender sinais de qualidade;
  - `timestamp`: o tempo absoluto não se repete no futuro.

  A função `matriz_rede` recusa a execução se alguma dessas colunas aparecer entre os preditores. `classe_atual` só entra na variante comparativa.
- **Candidatos** (`grade_rede`), além dos baselines classe majoritária e persistência:
  - 4 regressões logísticas: C = 0,1 ou 1, com ou sem pesos por classe;
  - 4 Random Forests: 150 árvores, folha mínima de 20 ou 100 exemplos, metade da amostra por árvore, com ou sem pesos;
  - 4 HistGradientBoosting: 200 iterações, 31 ou 63 folhas, com ou sem pesos.
- **Escolha:** maior macro-F1 na validação, com desempate pelo recall de FALHA. Venceu a Random Forest com folha 20 e sem pesos: 0,7628, contra 0,7618 do melhor HistGradientBoosting, um empate prático.
- **Duas saídas diferentes:**
  - `classe_prevista` é a classe de maior probabilidade;
  - `alerta_rede` é verdadeiro quando `score_alerta_rede` = P(RISCO) + P(FALHA) ≥ 0,3173. Esse limiar maximiza o F1 de "não OK" na validação.

  As duas podem discordar: uma medição pode ter OK como classe mais provável e ainda gerar alerta (exemplo na seção 6.1).

5.4 Tarefa lavagem

Objetivo: ordenar as transações para que a lavagem apareça no topo da fila de investigação.

- **Alvo:** `Is Laundering` (0 ou 1).
- **Atributos:** os 49 da Tabela 8, todos calculados na janela [t − w, t).
- **Excluídos:**
  - identificadores de contas e bancos, para não decorar contas;
  - horário e calendário, cujo padrão é artefato do gerador;
  - rótulos de transações passadas;
  - arquivos de contas e de padrões (seção 3.2);
  - a cauda de 11 a 18/09.
- **Preparação:** remoção de colunas constantes. Só para a regressão logística, também imputação, log com sinal e padronização.
- **Amostragem do treino:** todos os 2.530 positivos e 20% dos negativos, com peso 5 nos negativos para manter a proporção efetiva. São 651.808 linhas ajustadas. Validação e teste mantêm a distribuição original.
- **Candidatos** (`grade_financeira`), além dos baselines prevalência e taxa de lavagem do formato:
  - 2 regressões logísticas (C = 0,1 ou 1);
  - 1 Random Forest (100 árvores, folha 50, metade da amostra);
  - 2 HistGradientBoosting (300 iterações, 31 ou 63 folhas, regularização L2 = 1).

- **Escolha:** maior AP (*average precision*) na validação, por ordenar bem uma classe rara. Venceu a Random Forest, com 0,4990.
- **Limiar:** o que maximiza o F1 da classe lavagem na validação: 0,0910. Não foram fornecidos custos de erro; com eles, o critério poderia ser outro.

5.5 Algoritmos utilizados

Algoritmo	Como funciona, em resumo	Papel no NodeWatch
Baselines (`DummyClassifier`, persistência, taxa do formato)	Respostas sem aprendizado: sempre a classe mais comum, repetir o estado atual ou usar a taxa de lavagem do formato no treino.	Régua mínima: um modelo só vale a pena se superar essas respostas.
Regressão logística	**Classificador supervisionado**, apesar do nome: não prevê um número contínuo. Modela o log da chance de cada classe como combinação linear dos atributos e é ajustado por máxima verossimilhança com regularização L2, pelo otimizador L-BFGS (`tol` = 10^{-6} desde 02/11). É multinomial na rede e binário na lavagem.	Referência linear. Nunca foi escolhida: na lavagem, AP de validação de 0,19 contra 0,50 das árvores, o que mostra que as interações entre atributos pesam.
Random Forest	Conjunto de árvores de decisão (Breiman, 2001). Cada árvore é treinada numa amostra sorteada dos dados e, em cada divisão, considera só parte dos atributos. A probabilidade final é a média das árvores. As árvores são independentes e podem ser treinadas em paralelo (*bagging*).	**Modelo escolhido** nas duas tarefas.
HistGradientBoosting	Gradient boosting (Friedman, 2001): árvores construídas em sequência, cada uma corrigindo os erros das anteriores. Agrupa os valores de cada atributo em até 255 faixas (histogramas) para acelerar, como o LightGBM (Ke et al., 2017).	Candidato. Empatou com a Random Forest na rede.
XGBoost	Outra implementação de gradient boosting, com regularização própria (Chen e Guestrin, 2016). É um método diferente da Random Forest: árvores em sequência, e não independentes.	**Não está implementado no NodeWatch.** Foi apenas avaliado pela auditoria de 02/11 em scripts isolados (seção 7.4).
NumPy	Biblioteca de vetores e matrizes, não um algoritmo de aprendizado.	Usada nas peças próprias: o motor de atributos da seção 4.4 e métricas como início de evento, falsos alertas por fluxo-hora, limiar de máximo F1, ECE, Brier e PSI. **Nenhum modelo foi implementado à mão com NumPy**; os modelos vêm do scikit-learn 1.9.1.

5.6 Implementação própria, referência e baseline

Três papéis diferentes, que costumam ser confundidos:

Papel	O que é	Exemplos no projeto
Implementação própria	Código escrito pela equipe que faz parte do sistema.	Motor de atributos *point-in-time*, métricas próprias (`src/ml/avaliacao.py`), motivos, algoritmos de grafos.
Implementação de referência	Versão simples, escrita de forma independente e sem otimização, usada **só para conferir** a implementação própria.	`testes/ml/referencia_financeira.py`: recalcula os atributos financeiros com filtros diretos, transação por transação. Os testes exigem que o motor vetorizado dê o mesmo resultado.

Papel	O que é	Exemplos no projeto
Baseline de regras	Regra simples usada como régua de desempenho.	Persistência e classe majoritária (rede); prevalência e taxa do formato (lavagem); regras extras testadas pela auditoria (seção 7.4).

Os estimadores do scikit-learn são bibliotecas de terceiros amplamente testadas. O projeto não os reimplementa; verifica os próprios cálculos e o protocolo em volta deles.

5.7 Inferência com o pipeline salvo

1. **Carregar** o artefato (`ModeloUnificado.carregar`). Nada é reajustado.
2. **Validar** a entrada contra o esquema em modo inferência: `alvo` e `split` são opcionais; `classe_atual` só é exigida pela variante que a usa.
3. **Preparar** com os parâmetros salvos:
   - na rede, as mesmas sentinelas e indicadores;
   - na lavagem, o histórico das contas vem do arquivo indicado em `--historico`. Só entram transações anteriores à primeira transação a pontuar. Transações nas primeiras 72 h sem histórico recebem `historico_incompleto` = verdadeiro.
4. **Prever** em lotes. Lotes de menos de 1.000 linhas usam uma única thread, o que deixa a previsão de uma linha cerca de 3 vezes mais rápida sem mudar o resultado.
5. **Aplicar o limiar** salvo e, na lavagem, anexar os motivos das transações em alerta.
6. **Relatar**, preservando ordem e identidade das linhas de entrada (`arquivo_origem`, `linha_origem`). Toda linha sai com `tipo_saida` = `previsao`. Na rede, o relatório mede a **deriva**: o PSI (*Population Stability Index*) de cada atributo contra os quantis do treino. PSI ≥ 0,25 gera aviso de que o desempenho medido no teste pode não valer para aquela entrada.

> **As pontuações não são probabilidades de fraude.**
>
> - `score_alerta_rede` estima a chance de a próxima medição não ser OK. No teste, essa estimativa ficou bem calibrada (ECE de 0,010), mas trata de qualidade de rede, não de fraude.
> - `score_lavagem` foi aprendido com subamostragem e pesos em dados sintéticos. Serve para ordenar; o próprio artefato registra que não é fraude confirmada nem probabilidade calibrada.

6 Exemplos completos

Os exemplos usam previsões já geradas pelo artefato de 02/11/2026 e versionadas em `docs/ml/exemplos/`. Os valores foram preservados. Os atributos financeiros do exemplo 6.2 foram recalculados com o mesmo artefato para este documento, e a pontuação obtida é idêntica à do arquivo versionado. Os rótulos verdadeiros foram consultados nos dados originais só para mostrar o desfecho. Nenhuma transação foi associada a fluxo de rede.

6.1 Exemplo de rede

1. **Registro recebido.** Linha 13 (`linha_origem`) do arquivo derivado `rede_teste_sem_rotulos.csv`, que contém as linhas do período de teste sem `alvo` e `split`, embaralhadas. Fluxo `7341_4169` (sonda 7341 → âncora 4169), `timestamp` 1791156173.

2. **Validação.** Todas as colunas obrigatórias presentes, sem ausentes nem infinitos, valores dentro das faixas do esquema.
3. **Preparação.** Houve RTT, então nenhuma sentinela: `sem_rtt_w3/w6/w12` = 0. `ttl_reset` é descartada.
4. **Atributos relevantes:**

Atributo	Valor	Leitura
`razao_rtt_p95`	1,481	RTT 48% acima do p95 de referência
`razao_rtt_p99`	1,273	acima do p99: pela regra, a medição atual é FALHA
`n_rtt_acima_p95_w3` / `w12`	1 / 2	pico quase isolado: só esta medição entre as 3 últimas
`perda_pct`, `razao_jitter_p95`	0 / 0,247	sem perda; jitter normal

5. **Modelo (Random Forest):** P(OK) = 0,467; P(RISCO) = 0,079; P(FALHA) = 0,454.
6. **Resultado:** `classe_prevista` = OK (maior probabilidade). Mas `score_alerta_rede` = 0,079 + 0,454 = 0,533 ≥ 0,3173, então `alerta_rede` = verdadeiro.
7. **Interpretação:** o modelo vê quase meio a meio se a próxima medição continua em FALHA. O alerta diz "vale olhar este fluxo"; não diz nada sobre fraude.
8. **O que aconteceu:** no `dataset_final.csv`, a próxima medição desse fluxo foi OK. A persistência teria previsto FALHA e errado. A classe prevista acertou, e o alerta foi um falso alerta. Isso ilustra a troca entre recall e alertas falsos controlada pelo limiar.

Tabela 9 — Outras duas linhas da mesma amostra

Fluxo (linha)	Sinais principais	P(OK) / P(RISCO) / P(FALHA)	Previsto / alerta	Próxima medição real
`7509_1612` (48)	RTT acima do p95 nas 3, 6 e 12 últimas medições	0,021 / 0,066 / 0,913	FALHA / sim	FALHA
`7354_4261` (1)	RTT no limite do p95 (1,001); jitter alto na janela de 6	0,181 / 0,772 / 0,047	RISCO / sim	RISCO

Fonte: `docs/ml/exemplos/previsoes_rede_amostra.csv`; entradas em `artefatos_ml/exemplos/rede_teste_sem_rotulos.csv` (local); rótulos em `dataset_final.csv`. A amostra são as 200 primeiras linhas do arquivo embaralhado e não é representativa.

6.2 Exemplo financeiro

1. **Registro recebido** (dado sintético do HI-Small, sem a coluna de rótulo): `2022/09/10 11:00`, banco 12, conta `8010D4440` → banco 132554, conta `80D31F5E0`; 916.645,83 US Dollar pagos e recebidos; formato ACH. É a linha com `id_transacao` = 2 em `previsoes_lavagem_amostra.csv`.
2. **Validação.** Cabeçalho exato do `hismall_v1` (sem `Is Laundering`, permitido na inferência), sem ausentes, valores finitos e não negativos.
3. **Preparação.** O histórico vem do arquivo original (`--historico`), cortado antes da primeira transação do arquivo pontuado. O valor já está em dólar (taxa 1).

4. **Atributos relevantes:**

Atributo	Valor	Leitura
`log_valor_usd`	13,73	cerca de 917 mil dólares
`n_env_origem_6h/24h/72h/total`	2 / 4 / 12 / 17	conta ativa nos últimos 3 dias
`z_valor_origem`	4,16	valor muito acima do habitual da conta
`n_rec_origem_24h` / `soma_rec_origem_usd_24h`	2 / 9.510	recebeu cerca de 9,5 mil dólares em 24 h
`razao_saida_entrada_24h`	2,00	enviou o dobro do que recebeu em 24 h
`min_desde_env_origem`	329	último envio há cerca de 5,5 h
`par_novo` / `dest_distintos_origem`	1 / 11	primeira transação para esse destino
`orig_distintas_destino`	6	destino já recebeu de 6 contas

5. **Modelo (Random Forest):** `score_lavagem` = 0,1813.

6. **Resultado:** 0,1813 ≥ 0,0910, então `alerta_lavagem` = verdadeiro. Motivos: "formato com taxa de lavagem no treino ≥ 5x a média; primeira transação entre origem e destino; valor atípico para a origem (|z| > 3)". A saída registra `sinais_utilizados` = financeiro e `cobertura_vinculo` = sem_vinculo.

7. **Interpretação:** a transação vai para a fila de investigação. Os três motivos podem ser conferidos nos dados; nenhum deles prova lavagem.

8. **O que diz o rótulo sintético:** `Is Laundering` = 0. Pela verdade do simulador, é um **falso positivo**. A transação das 05:31 do mesmo dia e da mesma origem (1.380,75 francos suíços, ACH, pontuação 0,1086) tem rótulo 1. Nove dos 17 envios anteriores dessa conta também têm rótulo 1, mas rótulos passados nunca entram como atributo, então o modelo não sabe disso.

Fonte: `docs/ml/exemplos/previsoes_lavagem_amostra.csv` e o artefato `lavagem-20261102T083253Z`. A amostra são as 200 primeiras transações de 10/09/2022 e não é representativa: 95 delas geraram alerta.

6.3 Exemplo do módulo de grafos

Entrada: `dados/exemplo_transacoes.csv`. A Figura 6 desenha o grafo; a saída abaixo foi obtida executando a CLI na versão documentada.

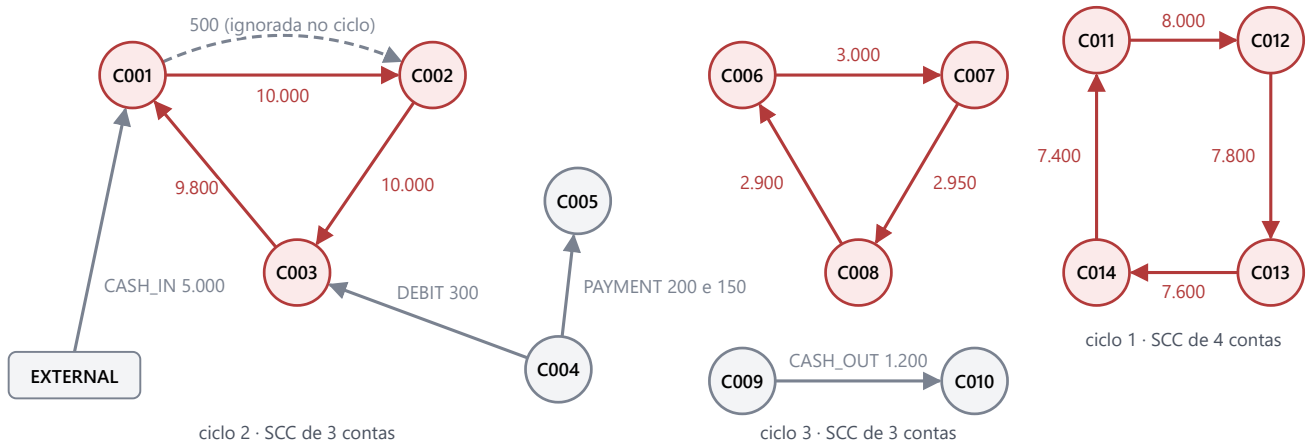


Figura 6 — Grafo do CSV de exemplo. Arestas vermelhas: ciclos encontrados. Entre C001 e C002 há duas transações paralelas (10.000 e 500).

```
python src/main.py --input dados/exemplo_transacoes.csv --scc --centralidade

Vértices (contas): 15 · Arestas (transações): 16
CICLO 1 [ALTO]: C011 -> C012 -> C013 -> C014 -> C011   valor 30.800,00   perda 600,00
CICLO 2 [ALTO]: C001 -> C002 -> C003 -> C001           valor 29.800,00   perda 200,00
CICLO 3 [ALTO]: C006 -> C007 -> C008 -> C006           valor  8.850,00   perda 100,00
Centralidade: as 10 contas dos ciclos têm risk_score 55 (40 por TRANSFER + 15 por 1 ciclo), tipo SUSPEITO
SCC suspeitos: {C011..C014} volume 30.800 · {C001, C002, C003} volume 30.300 · {C006, C007, C008} volume 8.850
```

Saída resumida: mesmos valores e mesma ordem impressos pelo programa, com os números reescritos no formato brasileiro.

Leitura. Os três anéis de TRANSFER têm prioridade ALTA e saem ordenados por valor. Duas observações mostram defeitos já conhecidos:

- O ciclo C001 → C002 → C003 vale 29.800, mas o volume interno do SCC é 30.300. O ciclo considera só a primeira transação entre C001 e C002 (10.000) e ignora a paralela de 500 (defeito A3).
- A "perda" é a diferença entre a primeira e a última aresta do caminho, não uma medida contábil.

A coluna `isFraud` marca a aresta que fecha cada ciclo, mas nenhum algoritmo a lê.

7 Avaliação e resultados

7.1 Como ler as métricas

VP = verdadeiros positivos, FP = falsos positivos, FN = falsos negativos. A classe "positiva" é a que interessa encontrar: lavagem, ou "não OK" na rede.

Métrica	Definição	Como ler aqui
Precisão	VP ÷ (VP + FP)	Dos alertas emitidos, quantos estavam certos.
Recall (sensibilidade)	VP ÷ (VP + FN)	Dos casos reais, quantos foram encontrados.
F1	2 × precisão × recall ÷ (precisão + recall)	Equilíbrio entre os dois; é o critério dos limiares.
Macro-F1	Média simples do F1 de OK, RISCO e FALHA	Cada classe pesa igual, mesmo FALHA, que tem só 3,7% das linhas.
Balanced accuracy	Média dos recalls das classes	Complemento do macro-F1.
Acurácia	Acertos ÷ total	Enganosa com classes desequilibradas: na rede, prever sempre OK dá 0,873.
Matriz de confusão	Tabela classe real × classe prevista	Mostra quais erros acontecem, não só quantos.
AP (*average precision*)	Área sob a curva precisão × recall, obtida percorrendo a lista ordenada pela pontuação: $\Sigma\,(R_n - R_{n-1}) \cdot P_n$	Qualidade da **ordenação**, sem limiar. O valor de referência é a prevalência (0,0011 na lavagem). **Não é acurácia:** prever sempre "não lavagem" dá acurácia de 99,89% e AP de apenas 0,0011.
Recall@k	Fração da lavagem do dia entre as k transações de maior pontuação	Simula uma equipe que investiga k casos por dia.
Falsos alertas por fluxo-hora	Falsos alertas ÷ (linhas × intervalo mediano, em horas)	Custo operacional dos alertas de rede.

Métrica	Definição	Como ler aqui
Início de evento	Passagem de OK para não OK entre medições consecutivas (até 480 s)	Se o modelo antecipa a piora, e não só a repete.
Brier e ECE	Erro quadrático da probabilidade; diferença média entre probabilidade prevista e frequência observada, em 10 faixas	Calibração: se "70%" acontece cerca de 70% das vezes.

7.2 Resultados da rede

Tabela 10 — Validação (escolha do modelo; ajuste só no treino)

Configuração	Macro-F1	Recall RISCO	Recall FALHA	Balanced acc.
Classe majoritária	0,3124	0,0000	0,0000	0,3333
Persistência (repete `classe_atual`)	0,7106	0,4826	0,7055	0,7109
Melhor regressão logística (C = 0,1, sem pesos)	0,5899	0,2924	0,3798	0,5535
Melhor HistGradientBoosting (31 folhas, sem pesos)	0,7618	0,4192	0,7240	0,7109
Random Forest com pesos por classe (folha 20)	0,7298	0,5573	0,7831	0,7597
Random Forest escolhida (folha 20, sem pesos)	**0,7628**	**0,3964**	**0,7383**	**0,7090**

Fonte: `docs/ml/resultados/treino_estado_rede.json`; grade completa (12 candidatos) em `docs/ml/RELATORIO_ML.md`, seção 1.2. Limiar escolhido: 0,3173.

Tabela 11 — Teste (05/10/2026, 46.105 medições, artefato congelado)

Referência	Macro-F1	Balanced acc.	Acurácia	Recall RISCO	Recall FALHA
Classe majoritária	0,311	0,333	0,873	0,000	0,000
Persistência	0,711	0,711	0,894	0,507	0,681
Modelo escolhido	**0,750**	**0,702**	**0,928**	**0,418**	**0,697**
Modelo + `classe_atual`	0,752	0,706	0,928	0,425	0,703

Matriz de confusão — modelo

Real \ previsto	OK	RISCO	FALHA
OK	39.925	201	124
RISCO	2.400	1.856	184
FALHA	273	156	986

Matriz de confusão — persistência

Real \ previsto	OK	RISCO	FALHA
OK	38.012	1.999	239
RISCO	1.980	2.251	209
FALHA	252	200	963

Tabela 12 — Alertas, eventos e fluxos inéditos (teste)

Medida	Persistência	Modelo
Alertas emitidos (limiar 0,3173 para o modelo)	5.861	4.621
Alertas corretos / falsos	3.623 / 2.238	3.674 / 947
Falsos alertas por fluxo-hora	0,73	0,31
Inícios de evento (2.229) antecipados	0%	4,5% (19% dos inícios de FALHA)
Acerto nas 4.879 transições de estado	0%	42%
Macro-F1 em fluxos inéditos (135 grupos)	0,674	0,681 ± 0,073
Macro-F1 em âncoras inéditas (30 grupos)	0,694	0,699 ± 0,083

Fonte: `docs/ml/resultados/avaliacao_estado_rede.json`; `docs/ml/RELATORIO_ML.md`, seção 1.3. Calibração de P(não OK) no teste: Brier 0,055 e ECE 0,010.

Leitura.

- O ganho sobre a persistência vem principalmente de **menos falsos alertas**: 947 contra 2.238.
- O recall de RISCO cai de 0,507 para 0,418.
- O modelo quase não antecipa o começo de um problema: 4,5% dos inícios de evento. A maior parte do acerto vem de estados que continuam e de eventos que terminam.
- Em fluxos e âncoras que não estavam no treino, a vantagem praticamente desaparece. O ganho depende de o modelo já conhecer o fluxo.

7.3 Resultados da lavagem

Tabela 13 — Validação (07–08/09/2022, distribuição original)

Configuração	AP	F1 no limiar	Precisão	Recall
Prevalência (constante)	0,0011	—	—	—
Taxa de lavagem do formato	0,0068	0,0151	0,0076	0,8591
Regressão logística (C = 0,1)	0,1895	0,2897	0,2282	0,3967
HistGradientBoosting (31 folhas)	0,4449	0,5017	0,6211	0,4208
Random Forest escolhida	**0,4990**	**0,5183**	**0,6384**	**0,4363**

Fonte: `docs/ml/resultados/treino_lavagem.json`. Limiar escolhido: 0,0910.

Tabela 14 — Teste (09–10/09/2022, 862.792 transações, 956 de lavagem)

Referência	AP	Precisão	Recall	F1	Falsos / 10 mil	VP	FP	FN
Prevalência	0,0011	0,001	1,000	0,002	9.988,9	956	861.836	0
Taxa do formato	0,0064	0,007	0,872	0,014	1.371,8	834	118.361	122
Modelo escolhido	**0,4640**	**0,627**	**0,407**	**0,494**	**2,7**	**389**	**231**	**567**

A AP cai de 0,499 na validação para 0,464 no teste, uma leve queda com o tempo. Com uma equipe que investigasse k transações por dia:

Dia	Transações	Lavagem	Recall@100	Recall@500	Recall@1000	Recall@5000
09/09/2022	654.467	514	0,18	0,44	0,54	0,71
10/09/2022	208.325	442	0,21	0,50	0,62	0,83

Resultados por tipologia e positivos sem padrão

O recall global de 0,407 mistura grupos muito diferentes. Dos 956 positivos do teste, 585 pertencem a uma tipologia de `HI-Small_Patterns.txt` e 371 (39%) não pertencem a nenhuma.

Tabela 15 — Recall por tipologia (no limiar)

Tipologia	Positivos	Detectados	Recall
BIPARTITE	44	24	0,55
CYCLE	55	33	0,60
FAN-IN	57	39	0,68
FAN-OUT	65	47	0,72
GATHER-SCATTER	127	85	0,67
RANDOM	43	24	0,56
SCATTER-GATHER	110	74	0,67
STACK	84	56	0,67
Sem padrão	**371**	**7**	**0,02**

Tabela 16 — Recall por início da tentativa

Grupo	Positivos	Detectados	Recall
Tentativa iniciada antes do teste	375	272	0,73
Tentativa nova (iniciada no teste)	210	110	0,52
Sem padrão	371	7	0,02

As tipologias têm recall entre 0,55 e 0,72. Os positivos sem padrão quase não são detectados. Em tentativas que já vinham do treino ou da validação, o recall é otimista, porque o modelo viu o começo do mesmo esquema.

Fonte: `docs/ml/resultados/avaliacao_lavagem.json`; `docs/ml/RELATORIO_ML.md`, seção 2.3. Na cauda (11–18/09), a AP de 0,969 é inflada pela base de 59% de lavagem e não representa operação.

7.4 Comparações feitas pela auditoria

A auditoria independente de 02/11/2026 comparou outras famílias de modelos e regras explícitas, sem alterar o NodeWatch. Os números abaixo vêm de `auditoria/ml_feedback_professora/resultados/TABELAS.md`, no repositório `AnaliseComplexidade_DeteccaoFraudes`. Na lavagem, foram medidos com os atributos **anteriores** à correção do `z_valor_origem`: a Random Forest de produção aparece com AP de teste 0,461, e não 0,464.

Tabela 17 — Rede (teste)

Modelo	Macro-F1	Treino	Tamanho
XGBoost (hist, CPU)	0,752	4,1 s	0,86 MB
HistGradientBoosting	0,750	4,5 s	0,97 MB
RF de produção	**0,750**	**12,2 s**	**17,6 MB**
RF com 50 árvores	0,751	5,8 s	5,9 MB
RF do notebook da professora	0,755	24,4 s	39,2 MB

Tabela 18 — Lavagem (teste)

Modelo	AP val.	AP teste	Treino
XGBoost (hist, CPU)	0,555	0,509	6,6 s
RF com busca de 4 tentativas	0,525	0,478	85,4 s
RF de produção	**0,495**	**0,461**	**42,7 s**
HistGradientBoosting	0,450	0,414	19,3 s
Regressão logística (tol antiga)	0,169	0,169	4,0 s

Rede: no bootstrap pareado por fluxo (135 fluxos, 2.000 reamostragens), a diferença XGBoost − RF é de +0,0016, com IC (intervalo de confiança) de 95% entre −0,0030 e +0,0057. É um empate. O XGBoost treina mais rápido e é menor, mas o HistGradientBoosting, que já está no projeto, e uma RF com menos árvores dão o mesmo resultado.

Lavagem: o XGBoost teve AP maior que a RF nos dois dias do teste. Com o mesmo número de tentativas de busca, a vantagem cai para cerca de 0,03. Três ressalvas:

- a AP de validação do XGBoost variou de 0,27 a 0,55 entre as 4 tentativas;
- a escolha usou a validação;
- o teste já havia sido consultado.

Por isso ele é **candidato** para uma confirmação futura, e não foi adotado.

Regras explícitas:

- Na rede, aplicar à medição atual a regra que gera o rótulo dá macro-F1 de 0,712, praticamente a persistência.
- Na lavagem, a melhor regra testada ("formato ACH e primeira transação do par") encontra 78% da lavagem, mas com precisão de 0,036: cerca de 230 falsos alertas por 10 mil transações, contra 2,7 do modelo atual (Tabela 14).

7.5 Reuso do teste e incerteza

> **O teste já foi consultado mais de uma vez.** O treino nunca leu rótulos do teste, e as escolhas foram feitas na validação. Mesmo assim, o período de teste foi usado:
>
> - na avaliação de 26/10/2026;
> - na comparação de 9 candidatos feita pela auditoria;
> - na reavaliação de 02/11/2026.
>
> Além disso, o protocolo foi desenhado depois de olhar a distribuição dos rótulos de todos os dias. Os números de teste, portanto, não são uma avaliação inédita.

Diferenças pequenas não devem ser lidas como melhoria comprovada. São exemplos o 0,750 contra 0,752 do modelo com `classe_atual` e o 0,461 contra 0,464 depois da correção do z. Antes de trocar o modelo de qualquer tarefa, o candidato deve ser congelado pela validação e medido **uma única vez** num período novo:

- **rede:** telemetria posterior a 05/10/2026;
- **lavagem:** uma replicação declarada, como o HI-Medium, já que o HI-Small não tem outro período limpo.

7.6 Testes automatizados

Em 02/11/2026, a `main` (commit 03b84d2) passou nos **88 testes** da suíte, executados com `pytest`: 39 do módulo de ML e 49 dos módulos de grafos.

Arquivo	Itens	O que verifica
`testes/ml/test_rede.py`	18	`alvo`, `split` e IDs nunca viram preditores; coluna ausente é rejeitada sem virar zero; valores inválidos são recusados; sentinelas aceitas; inferência sem rótulos; duplicatas e conflitos em vários CSVs; ordem e identidade preservadas; previsões idênticas depois de salvar e carregar; avaliação com artefato congelado.
`testes/ml/test_financeiro.py`	8	Atributos não mudam ao acrescentar transações futuras; transações do mesmo minuto não se enxergam; câmbio; amostragem; leitura de `.csv` e `.zip`; padrões lidos mas nunca usados como atributo.
`testes/ml/test_referencia_financeira.py`	8	Motor vetorizado igual à força bruta (3 sementes e casos de fronteira); z limitado e preciso; previsão de poucas linhas com uma thread; tolerância da regressão logística.
`testes/ml/test_integracao.py`	5	Fusão indisponível lista pendências; sem vínculo, nenhum sinal é associado; junção "as of" só usa sinal já emitido.
`testes/test_cycle_detection.py`, `test_scc.py`, `test_centralidade.py`, `test_exportador.py`	27	Ciclos conhecidos, grafo vazio, DAG (grafo acíclico) sem ciclos, laços; SCC e volume interno; graus, volumes e `risk_score`; seções e arquivo do JSON.
`testes/test_fan_detection.py`, `test_escala.py`	22	Fan-out e fan-in por janela; ciclo muito longo sem estouro de recursão; volume interno com vários SCC; laços dentro de SCC maiores.

> **Testes de software não medem qualidade preditiva.** Os testes usam dados pequenos e sintéticos e verificam se o código faz o que promete: contratos, ausência de vazamento, determinismo e equivalência com a referência. A qualidade do modelo é medida pela avaliação em períodos separados (seções 7.2 a 7.4). Uma suíte toda verde não garante um bom modelo. Os testes de grafos também não pegam os defeitos A1 a A4, que alteram resultados (seção 9.2).

8 Uso e reprodução

Todos os comandos rodam a partir da raiz do repositório, com o ambiente virtual ativado.

8.1 Instalação

```
python -m venv .venv
.venv\Scripts\Activate.ps1                      # Windows (no Linux/macOS: source .venv/bin/activate)
python -m pip install -r requirements.txt       # grafos, painel e Sentinel AI
python -m pip install -r requirements-ml.txt    # módulo de ML (versões fixadas)
```

O treino documentado usou Python 3.14.0. O `requirements-ml.txt` fixa numpy 2.5.4, pandas 3.1.0, scikit-learn 1.9.1, joblib 1.6.0 e pyarrow 26.0.0. O XGBoost não faz parte das dependências.

8.2 Módulo de grafos e painel

```
python src/main.py --input dados/exemplo_transacoes.csv                 # ciclos
python src/main.py --input dados/exemplo_transacoes.csv --scc --centralidade
python src/main.py --input dados/exemplo_transacoes.csv --export-json --output minha_analise
python -m pytest testes/ -v
```

Painel e Sentinel AI:

1. copie `.env.example` para `.env` e preencha `GEMINI_API_KEY`; o `.env` não vai para o git;
2. rode `start_presentation.ps1` (Windows) ou `start_presentation.sh`, ou ainda `python src/sentinel_ai.py`;
3. abra `http://127.0.0.1:5000/`.

O painel mostra os dados de exemplo embutidos; não aceita envio de arquivos.

8.3 Módulo de ML

Copie `ml_config.exemplo.toml` para `ml_config.local.toml` (ignorado pelo git) e ajuste os caminhos de `[rede].arquivos` e `[financeiro].transacoes`, além de `contas` e `padroes`, que são opcionais. Os arquivos originais nunca são alterados.

Comando	O que faz
`python -m src.ml.cli auditar --fonte todas`	Audita os dados e grava JSON e MD em `artefatos_ml/auditoria`.
`python -m src.ml.cli treinar --tarefa todas`	Ajusta no treino, escolhe na validação e salva o artefato.
`python -m src.ml.cli avaliar --tarefa todas`	Avalia uma vez no teste, com o artefato congelado.
`python -m src.ml.cli prever --tarefa estado_rede --entrada novo.csv --saida prev.csv`	Inferência de rede; aceita vários arquivos em `--entrada`.

Comando	O que faz
`python -m src.ml.cli prever --tarefa lavagem --entrada trans.csv --saida prev.csv --historico HI-Small_Trans.zip`	Inferência de lavagem com histórico das contas.
`python -m src.ml.cli tarefas`	Lista as cabeças, o status e o que falta para a fusão.

`python -m src.ml.cli exemplos`
`python -m src.ml.cli relatorio`
`python -m src.ml.cli documentar`
	Gera entradas de exemplo e suas previsões; gera `RELATORIO_ML.md`; gera o dicionário de dados e os esquemas.

`python -m pytest testes/ml`	Testes do módulo de ML.

8.4 Como preparar um arquivo compatível

- **Rede (`rede_v1`):**
  - colunas obrigatórias: `id_fluxo`, `anchor_id`, `probe_id`, `timestamp` e os 21 preditores da Tabela 7;
  - `alvo`, `split` e `classe_atual` são dispensáveis na inferência;
  - use −1 apenas onde ele é sentinela documentada;
  - vários arquivos precisam ter o mesmo cabeçalho;
  - as colunas `arquivo_origem` e `linha_origem` são reservadas.
- **Lavagem (`hismall_v1`):**
  - o cabeçalho deve ser exatamente o do HI-Small, com `Account` duas vezes;
  - `Is Laundering` é opcional na inferência;
  - aceita `.csv` ou `.zip` com um único CSV;
  - informe em `--historico` as transações anteriores, de pelo menos 72 h.
- **Grafos:** CSV com cabeçalho contendo `nameOrig`, `nameDest` e `amount`. `type`, `step` e `isFraud` são opcionais.

8.5 Como carregar o modelo e ler as saídas

```
from pathlib import Path
from src.ml.modelos import ModeloUnificado

modelo = ModeloUnificado.carregar(Path("artefatos_ml"))
print(modelo.tarefas_disponiveis())   # {'fusao_rede_transacao': 'indisponivel', 'estado_rede': 'treinada', ...}
saida, relatorio = modelo.prever("estado_rede", ["novo.csv"])
saida_l, rel_l = modelo.prever("lavagem", "trans.csv", historico="HI-Small_Trans.zip")
```

Tabela 19 — Colunas das previsões

Coluna	Significado
`arquivo_origem`, `linha_origem`	De onde veio a linha; a ordem de entrada é preservada.
`classe_prevista`, `prob_OK/RISCO/FALHA`	(rede) Classe mais provável e as três probabilidades estimadas.
`score_alerta_rede`, `alerta_rede`	(rede) P(RISCO) + P(FALHA) e se passou do limiar.
`horizonte`, `modelo_rede_treinado_ate`	(rede) "proxima_medicao"; fim do período de treino do modelo.
`score_lavagem`, `alerta_lavagem`	(lavagem) Pontuação e se passou do limiar.
`motivos_verificaveis`	(lavagem) Fatos conferíveis, só nas linhas em alerta (seção 4.5).

Coluna	Significado
`historico_incompleto`	(lavagem) Menos de 72 h de histórico antes da transação.
`sinais_utilizados`, `cobertura_vinculo`, `pontuacao_fraude_financeira`	(lavagem) Sempre "financeiro", "sem_vinculo" e "indisponivel" na versão atual.
`limiar_alerta`, `versao_modelo`, `tipo_saida`	Limiar usado, versão do artefato e "previsao".

O arquivo `.relatorio.json` traz as linhas previstas, a cobertura e, na rede, a deriva (PSI por coluna e taxa de sentinelas na entrada contra o treino). O `RELATORIO_ML.md` reúne auditoria, seleção e teste das duas tarefas, gerado só a partir dos resultados salvos.

8.6 Colunas ausentes e dados inválidos

Situação	O que acontece
Módulo de ML	
Coluna obrigatória ausente	`ErroEsquema` com a lista das colunas. Nada é preenchido com zero.
NaN, infinito, valor fora da faixa ou do domínio	`ErroEsquema` com a contagem por coluna. A sentinela −1 é aceita onde documentada.
Cabeçalhos diferentes entre arquivos de rede; mesmo arquivo informado duas vezes	`ErroEsquema`; `ErroDados`.
Chave `id_fluxo` + `timestamp` repetida	No treino, cópias idênticas são removidas, e cópias com valores diferentes geram erro (ou são excluídas, com `conflitos = "excluir"`). Na inferência, são só relatadas.
HI-Small com cabeçalho inesperado, ausentes, valores negativos ou não finitos, rótulo fora de {0, 1}	`ErroEsquema`.
Moeda sem taxa de câmbio estimada	Valor em USD ausente e `sem_taxa_cambio` = 1; a previsão segue.
Artefato inexistente ou de outra versão do scikit-learn	`FileNotFoundError` ("Rode 'treinar' antes"); aviso de versão.
Pedido da tarefa de fusão	`FusaoIndisponivel` com a lista de requisitos; a CLI sai com código 2.
Módulo de grafos	
Arquivo inexistente; cabeçalho vazio ou sem colunas obrigatórias	A CLI sai com código 2 ou 3 e mensagem de erro.
`amount` não numérico	`CsvFormatError` com o número da linha.
Linha sem origem ou destino	**Ignorada sem aviso** (defeito A7, aberto).
`amount` NaN, infinito ou negativo	**Aceito** e propagado para volumes e pontuações (A7, aberto).
`step` inválido	Vira ausente; mas `step` = `inf` derruba a leitura com `OverflowError` (A6, aberto).

9 Limitações e próximos passos

9.1 Limitações verificadas

Tema	Limitação	Consequência
Dependência dos dados	A lavagem foi aprendida e medida só no HI-Small sintético; parte do sinal (ACH) pode ser artefato do gerador.	Desempenho em dados reais é desconhecido.
Integração	Não há vínculo entre transações e rede.	Nenhuma pontuação de fraude usa sinais de rede.
Rótulo de rede	O alvo é uma regra de limiares sobre a próxima medição.	O modelo aprende a antecipar a regra; quase não prevê o início de eventos (4,5%).
Generalização (rede)	Em fluxos e âncoras inéditos, o ganho sobre a persistência quase some.	Novos fluxos exigem cautela ou retreino.
Cobertura (lavagem)	Positivos sem padrão: recall de 0,02. Tentativas novas: 0,52.	O recall global (0,41) superestima a capacidade em esquemas desconhecidos.
Falsos alertas	Rede: 947 falsos em 46.105 medições (0,31 por fluxo-hora). Lavagem: 231 em 862.792 (2,7 por 10 mil). Grafos: ciclos, SCC e hubs também surgem em operações legítimas.	Todo alerta precisa de checagem humana.
Avaliação	O teste foi reutilizado (seção 7.5).	Os números de teste não são inéditos.
Grafos	Análises estruturais ignoram a ordem no tempo; a enumeração de ciclos tem pior caso exponencial; o painel usa dados embutidos.	Ciclos "impossíveis" no tempo podem aparecer; arquivos grandes podem ser lentos.
Desempenho on-line	Os índices de atributos financeiros são reconstruídos sobre todo o histórico a cada chamada.	Pontuar uma transação isolada contra 5 milhões de linhas leva cerca de 12 s, quase tudo em atributos.

9.2 Pendências das auditorias

Duas auditorias examinaram o projeto: a de prontidão para ML (26/10/2026), que listou os defeitos A1 a A7 do módulo de grafos, e a auditoria de ML de 02/11/2026.

Item	Situação	Observação
A1 — prioridade não entra na ordenação dos ciclos	aberto	`cycle_detection.py`
A2 — `risk_score` não determinístico em empate de tipo	aberto	`centralidade.py`
A3 — arestas paralelas ignoradas no valor do ciclo	aberto	Visível no exemplo da seção 6.3
A4 — `has_cycle` recursivo estoura em caminhos longos	aberto	`find_cycles` já é iterativo
A5 — `add_edges_from` descarta `step`	aberto	`graph.py`
A6 — `step = inf` derruba a leitura	aberto	`file_reader.py`
A7 — `amount` inválido aceito; linhas sem conta descartadas sem aviso	aberto	`file_reader.py`
Divergência em `razao_saida_entrada_24h`	aberto	Acumular somas por conta
Motivos com p99 dominado por contas de alto volume	aberto	Seção 4.5

Item	Situação	Observação
Índice incremental de atributos para uso on-line	aberto	Antes de qualquer uso transação a transação
Confirmação em período intocado; decisão sobre XGBoost	aberto	Seção 7.5
Cancelamento e piso de variância em `z_valor_origem`	resolvido	02/11/2026, com testes
Tolerância da regressão logística (10^{-6})	resolvido	AP de validação de 0,165 para 0,19
Uma thread em previsões de menos de 1.000 linhas	resolvido	1 linha: 27,7 ms → 8,8 ms
Testes de força bruta; recall por tipologia e tentativa	resolvido	Seções 7.3 e 7.6

9.3 Como trocar modelos no futuro

A estrutura modular permite substituir o modelo de uma tarefa sem mexer na outra, inclusive por uma rede neural. Para isso, o novo modelo precisa cumprir o mesmo contrato de uma `CabecaClassificacao`:

- um pipeline com `predict_proba` e `classes_`;
- a lista exata de colunas de entrada e o limiar de alerta;
- uma fábrica registrada em `grade_rede` ou `grade_financeira`. A função `recriar_estimador`, que hoje conhece só três famílias, também precisa ser estendida.

Uma rede neural para dados tabulares, como um perceptron multicamada, poderia entrar como mais uma candidata. Uma rede neural em grafos exigiria algo maior: um novo codificador, que monte o grafo de contas respeitando o tempo.

> **Trocar de modelo não garante melhora.** Em dados tabulares como estes, árvores combinadas costumam ser difíceis de superar; na rede, quatro famílias já empatam. Qualquer substituição deve seguir o mesmo protocolo: ajuste no treino, escolha e limiar na validação, medição única num período intocado. Também precisa considerar custo de treino, latência, tamanho e explicabilidade.

Os próximos passos de maior valor não são de modelo:

1. obter dados com vínculo verificável e rótulos com data de confirmação, o que destravaria a fusão;
2. reservar um período novo para confirmação;
3. corrigir os defeitos A1 a A7 com testes de regressão;
4. tornar incremental o cálculo de atributos.

10 Glossário

Termo	Significado
ACH	*Automated Clearing House*: transferência eletrônica em lote; formato que concentra 87% da lavagem no HI-Small.
AML	*Anti-Money Laundering*: prevenção à lavagem de dinheiro.
AP	*Average precision*: qualidade da ordenação de uma classe rara (seção 7.1).
Artefato	Arquivo com o modelo treinado e seus metadados (`modelo_unificado.joblib`).
Baseline	Resposta simples usada como régua mínima de desempenho.
Bootstrap pareado	Reamostragem de grupos (aqui, fluxos) para estimar a incerteza da diferença entre dois modelos.

Termo	Significado
Cabeça	Componente do `ModeloUnificado` responsável por uma tarefa.
Calibração	Concordância entre probabilidade prevista e frequência observada (Brier, ECE).
Codificador	Componente que guarda o que foi aprendido sobre uma fonte de dados.
CPU, GPU	Processador central; processador gráfico. Nenhum modelo do projeto exige GPU.
Deriva	Mudança na distribuição dos dados de entrada em relação ao treino (medida pelo PSI).
DFS	*Depth-First Search*: busca em profundidade num grafo.
ECE	*Expected Calibration Error*: erro médio de calibração em faixas de probabilidade.
Fan-out / fan-in	Uma conta que envia para (ou recebe de) muitas contas distintas em pouco tempo.
Fluxo	Par sonda → âncora cujas medições formam uma série temporal.
Jitter	Variação do atraso entre pacotes.
Layering	Fase da lavagem em que o dinheiro passa por várias contas para esconder a origem.
Limiar	Valor de pontuação a partir do qual se emite alerta.
LLM	*Large Language Model*: modelo de linguagem; usado só pelo Sentinel AI, por API externa.
Multigrafo direcionado	Grafo com arestas orientadas, que pode ter várias arestas entre o mesmo par de vértices.
Persistência	Baseline que prevê que o próximo estado repete o atual.
Point-in-time	Cálculo que só usa informação disponível antes do instante da previsão.
PSI	*Population Stability Index*: medida de deriva entre duas distribuições.
RTT	*Round-Trip Time*: tempo de ida e volta de um pacote.
SCC	*Strongly Connected Component*: grupo de vértices em que todos alcançam todos.
Sentinela	Valor especial (aqui, −1) que significa "não medido", e não um número real.
TTL	*Time To Live*: contador de saltos de um pacote; sua variação indica mudança de rota.
USD	Dólar americano, moeda base dos atributos financeiros.
Vazamento (*leakage*)	Uso, no treino ou nos atributos, de informação que não estaria disponível no momento da previsão.

11 Referências

Referências técnicas

ALTMAN, E. et al. Realistic Synthetic Financial Transactions for Anti-Money Laundering Models. In: *Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track)*, 2023. arXiv:2306.16424.

BREIMAN, L. Random Forests. *Machine Learning*, v. 45, n. 1, p. 5–32, 2001.

BRIER, G. W. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, v. 78, n. 1, p. 1–3, 1950.

CHEN, T.; GUESTRIN, C. XGBoost: A Scalable Tree Boosting System. In: *Proceedings of the 22nd ACM SIGKDD*, p. 785–794, 2016.

CORMEN, T. H. et al. *Introduction to Algorithms*. 3. ed. MIT Press, 2009. Cap. 22 (busca em profundidade e componentes fortemente conectados).

EFRON, B.; TIBSHIRANI, R. J. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

FRIEDMAN, J. H. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, v. 29, n. 5, p. 1189–1232, 2001.

HARRIS, C. R. et al. Array programming with NumPy. *Nature*, v. 585, p. 357–362, 2020.

HOSMER, D. W.; LEMESHOW, S.; STURDIVANT, R. X. *Applied Logistic Regression*. 3. ed. Wiley, 2013.

JOHNSON, D. B. Finding all the elementary circuits of a directed graph. *SIAM Journal on Computing*, v. 4, n. 1, p. 77–84, 1975.

KE, G. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In: *Advances in Neural Information Processing Systems 30*, 2017.

LIU, D. C.; NOCEDAL, J. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, v. 45, p. 503–528, 1989.

LOPEZ-ROJAS, E. A.; ELMIR, A.; AXELSSON, S. PaySim: A financial mobile money simulator for fraud detection. In: *28th European Modeling and Simulation Symposium (EMSS)*, 2016.

NAEINI, M. P.; COOPER, G. F.; HAUSKRECHT, M. Obtaining Well Calibrated Probabilities Using Bayesian Binning. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.

PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.

SAITO, T.; REHMSMEIER, M. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, v. 10, n. 3, e0118432, 2015.

SHARIR, M. A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, v. 7, n. 1, p. 67–72, 1981.

SIDDIQI, N. *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Wiley, 2006 (índice de estabilidade populacional).

Arquivos do projeto usados como fonte

Arquivo	Conteúdo
`README.md`	Visão do MVP, integrantes, comandos de grafos e painel.
`src/main.py`, `src/algoritmos/`, `src/grafo/`, `src/leitura/`	Módulo de grafos (seções 4.2 e 6.3).
`src/sentinel_ai.py`, `interface/index.html`	Painel e assistente.
`src/ml/*.py`	Módulo de ML: esquemas, ingestão, auditoria, preparação, modelos, experimentos, avaliação, previsão, integração, relatório e CLI.
`testes/`, `testes/ml/`	Suíte de 88 testes (seção 7.6).
`docs/ml/README.md`, `DICIONARIO_DADOS.md`, `CONTRATO_INTEGRACAO.md`	Instalação, dicionário e contrato de integração.
`docs/ml/RELATORIO_ML.md`, `docs/ml/notas_relatorio.md`	Resultados e leitura da execução de 02/11/2026.
`docs/ml/resultados/*.json`, `docs/ml/exemplos/*`	Métricas, metadados do artefato e amostras de previsão.
`AnaliseComplexidade_DeteccaoFraudes`: `auditoria/ml_feedback_professora/`	Auditoria de ML de 02/11/2026: relatório, scripts e `resultados/TABELAS.md`.
`AnaliseComplexidade_DeteccaoFraudes`: `auditoria/AUDITORIA_NODEWATCH_ML_2026.md`	Auditoria de prontidão de 26/10/2026 (defeitos A1 a A7). Arquivo local, não versionado.